



Python for Beginners

PASS BY VALUE VS PASS BY REFERENCE

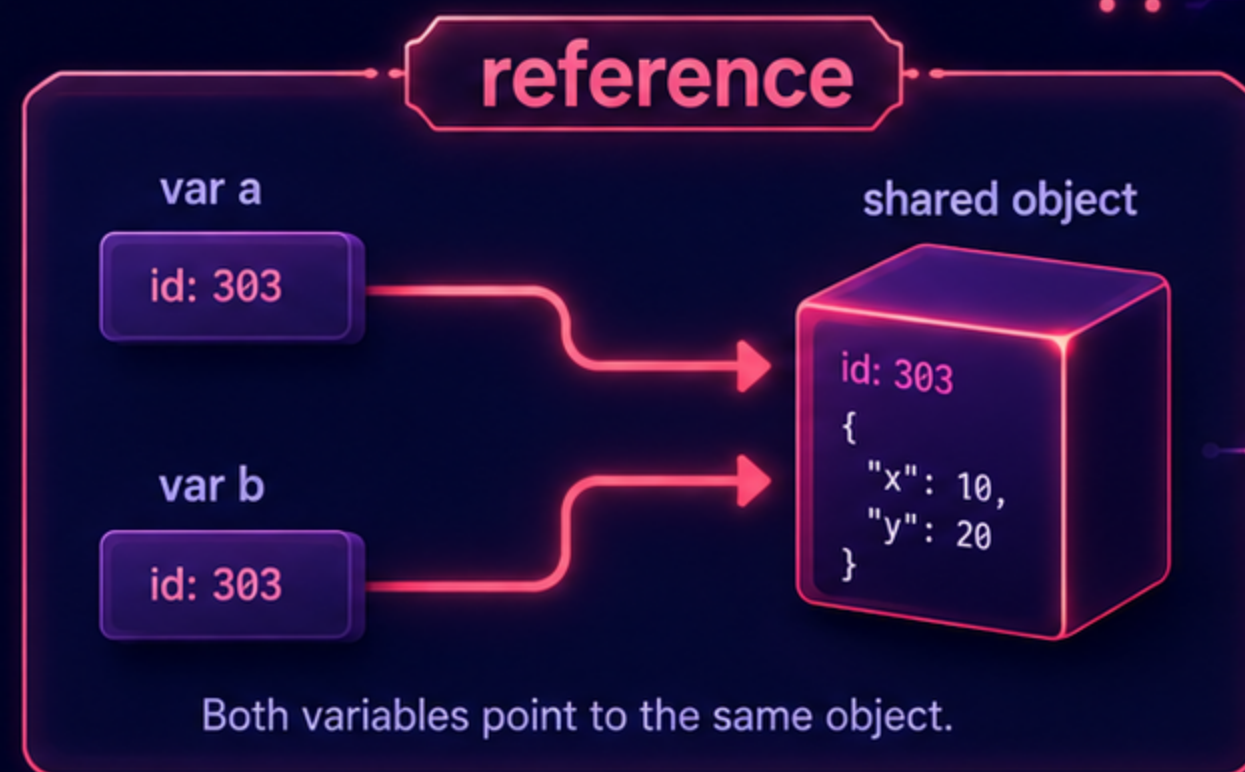
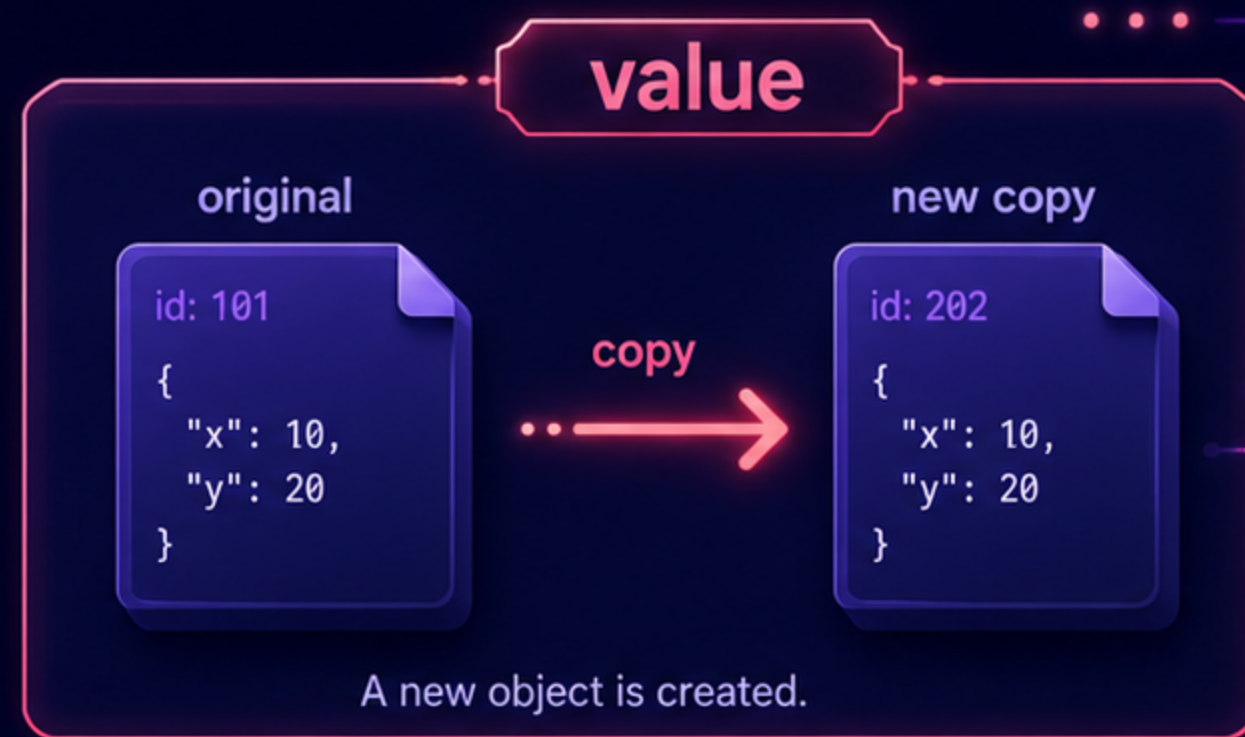
How Python REALLY passes variables to functions

```
def func(x):  
    x['x'] = 99  
    x = {'x': 123}  
    return x
```



Code and Debug

Python for Beginners



The Question



example.py

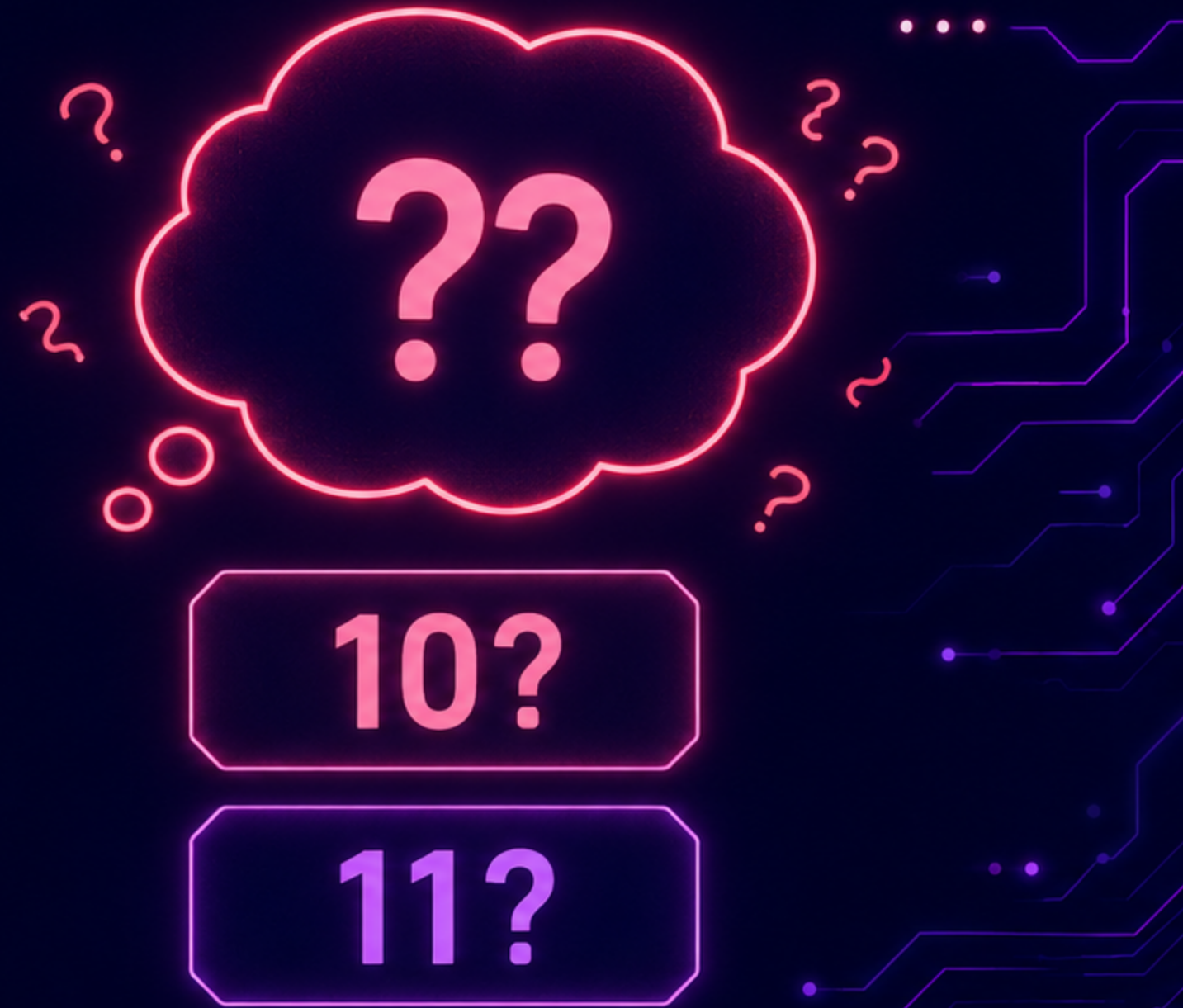
```
def increment(x):  
    x = x + 1  
  
num = 10  
increment(num)  
print(num)    # What prints? 10 or 11?
```

When you pass a variable to a function,
what happens to it?

This is one of the most confusing topics for
Python beginners because Python's behavior is unique.



Watch Out: The answer depends on whether you're passing a mutable or immutable object. Let's unpack it step by step.



WHAT OTHER LANGUAGES DO

Languages like C, Java, C# distinguish two classical approaches:

PASS BY VALUE

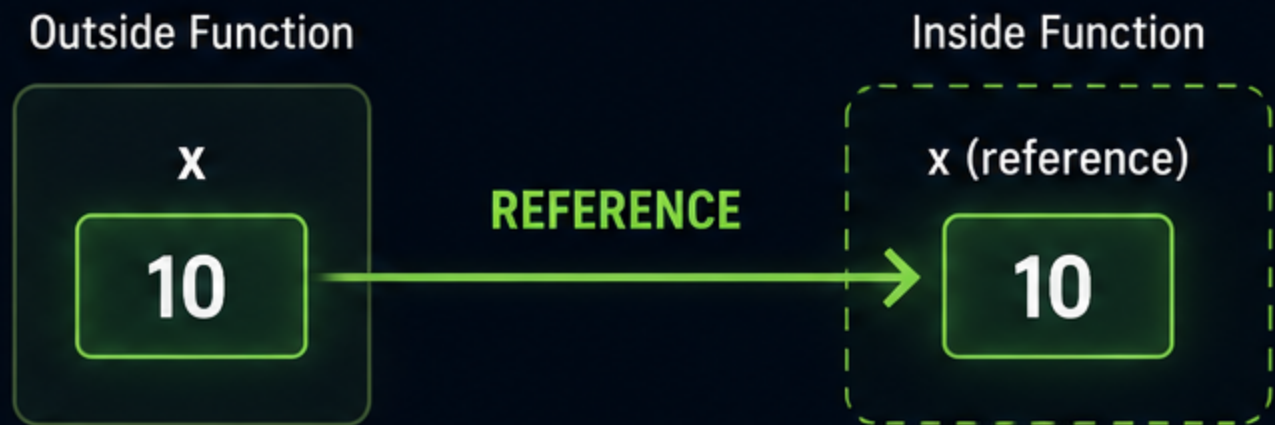
Function gets a copy of the variable.
Changes inside the function don't affect the original.



Classic in C for primitive types

PASS BY REFERENCE

Function gets a **reference** to the variable.
Changes inside the function affect the original.



Classic in C++ with & references



PASS BY OBJECT REFERENCE – ??

Python does neither – strictly speaking.
Python uses something called
“**pass by object reference**” (also known as “**call by sharing**”).
Let's see what that really means.



Python with Immutable Objects

Let's run our earlier code with an integer (immutable):

```
example.py

def increment(x):
    x = x + 1
    print(f"Inside function: {x}")

num = 10
increment(num)
print(f"Outside function: {num}")
```

Output:

```
Inside function: 11
Outside function: 10
```

What happened?

- 1. `x = x + 1` creates a new integer object (11)
- 2. The local `x` now points to this new object
- 3. The original `num` outside still points to 10
- 4. Integers are immutable - they can't be changed in place



Note: It looks like pass by value - but that's not quite what's happening.

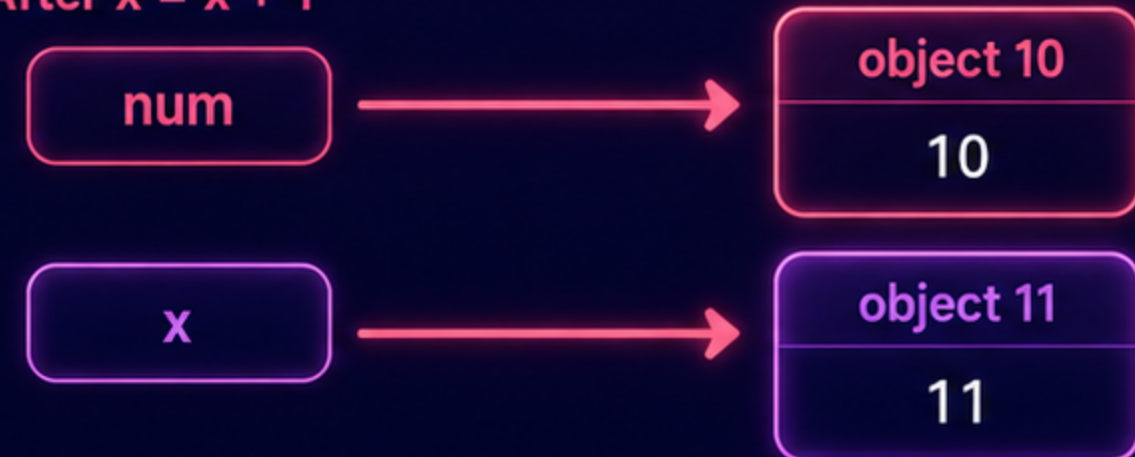
Outside function



Function starts



After `x = x + 1`



Python with Mutable Objects

Now try the same with a list (mutable):

```
def add_item(lst):  
    lst.append(99)  
    print(f"Inside function: {lst}")  
  
items = [1, 2, 3]  
add_item(items)  
print(f"Outside function: {items}")
```

Output:

```
Inside function: [1, 2, 3, 99]  
Outside function: [1, 2, 3, 99]
```

What happened?

- 1. The list is mutated in place by `.append()`
- 2. Both `lst` (inside) and `items` (outside) point to the SAME list object
- 3. Changes are visible from everywhere



Note: This looks like pass by reference - but again, not quite.

Outside function

items

list object
[1, 2, 3]

Function starts

items

lst

list object
[1, 2, 3]

After `.append(99)`

items

lst

list object
[1, 2, 3, 99]

THE REAL MODEL: PASS BY OBJECT REFERENCE

Python passes the **reference to the object** — not a copy of the variable, and not the variable itself.



THE RULE:

- Variables in Python are **names that point to objects**
- When you pass a variable, Python **passes the reference**
- Inside the function, the parameter becomes a new local name pointing to the **SAME object**
- What you can **DO** with it depends on whether the object is mutable or immutable



QUICK TIP:

Forget “pass by value” and “pass by reference” entirely. Just remember: functions get a reference to the **same object** — and the object’s mutability determines what changes are visible outside.

OUTSIDE FUNCTION

Name

items

items is a **name** that points to the **list object**

LIST OBJECT



[1, 2, 3]

This is the **actual** list object in memory.

INSIDE FUNCTION

Local Name (Parameter)

lst

lst is a **new local name** that points to the **SAME** list object



TWO NAMES, ONE OBJECT



Rebinding vs Mutating (The Key Insight)

This is the key distinction that explains everything.

Rebinding

```
def rebind(lst):  
    lst = [99, 99, 99]  # local name now points to a NEW list  
  
items = [1, 2, 3]  
rebind(items)  
print(items)  # [1, 2, 3] - unchanged!
```

outside

items

list object A

[1, 2, 3]

inside function

lst

list object B

[99, 99, 99]

Assignment rebinds the local name to a NEW object.

Mutating

```
def mutate(lst):  
    lst.append(99)  # modifies the SAME list object  
  
items = [1, 2, 3]  
mutate(items)  
print(items)  # [1, 2, 3, 99] - changed!
```

outside

items

shared list object

[1, 2, 3]

inside function

lst

after mutation

append(99)

same object

[1, 2, 3, 99]

Mutation changes the shared object in place.



Quick Tip

Assignment (=) inside a function rebinds only the local name.

Mutation methods like `.append()`, `.sort()`, or `lst[i] = x` modify the shared object in place.